AI Prototype Demonstration and Deployment Planning

Machine Learning-Based Multi-Class Network Intrusion Detection | Group 19 | FUSE AI-201

MODULE 6 | PRACTICAL PROTOTYPE DEMONSTRATION AND DEPLOYMENT PLANNING

Project team: Adugna Necho Mulatu; Aminadab Tewhibo Mihret; Hussen Yasin Mohammed; Abdulfetah Hayerdin Mohammed; Nebiyu Mesfin Fikade.

Prototype release status: Source and deployment package prepared. The trained model binary was not included in the supplied files; the training/export script must be run and the artifact tested before a public prediction service can be called live.

1. Executive summary

Group 19 developed a five-class network-intrusion classification prototype using the NSL-KDD benchmark. It maps connection records to Normal, Denial of Service (DoS), Probe, User-to-Root (U2R), and Remote-to-Local (R2L). The package includes a reproducible training/export script, input validation and inference helpers, a Streamlit CSV interface, an optional FastAPI endpoint, tests, sample input, and deployment/maintenance documentation.

The system demonstrates how a user can submit a connection-record CSV and receive a predicted class, confidence score, threat-severity label and class probabilities. It is a benchmark-data prototype, not a live packet-capture system, firewall or production security monitoring service.

2. Problem, dataset and model

Network operators need to identify suspicious traffic patterns while distinguishing benign activity from different attack categories. This project explores supervised multi-class classification on NSL-KDD connection-level features. The supplied Module 5 notebook loads 125,973 KDDTrain+ rows and 22,544 KDDTest+ rows, each represented by 41 model features plus label/metadata columns. Detailed labels are consolidated into five target classes.

A stratified 17.65% validation split was formed from KDDTrain+ (103,738 training rows; 22,235 validation rows; random_state=42). KDDTest+ was reserved for final evaluation. The selected estimator is a Random Forest within an imbalanced-learn pipeline: SMOTENC for minority training examples, median imputation and standard scaling for numeric fields, and imputation plus one-hot encoding for protocol_type, service and flag. The three-fold GridSearchCV selected 250 trees, maximum depth 20 and minimum split 5 using Macro-F1; best cross-validation Macro-F1 was 0.9019.

3. Evaluation and evidence

Metric	Validation split	Independent KDDTest+
Accuracy	99.8876%	74.6274%
Macro precision	97.1483%	80.0803%
Macro recall	99.4603%	48.8729%
Macro F1	98.2503%	51.5480%

The independent-test class recalls were Normal 0.97, DoS 0.77, Probe 0.54, U2R 0.10 and R2L 0.06. The highest one-versus-rest false-positive rate was 0.408166 for Normal. All five proposal thresholds recorded in the notebook were

marked Fail on KDDTest+. The difference between validation and independent-test results is material and must be discussed, not concealed.

The notebook measured 0.1190 ms per record for a 1,000-record prediction batch in its original Colab environment. This is a runtime-specific observation and not a service-level guarantee. The notebook also includes a confusion matrix, classification report, class-level false-positive rates, ROC-AUC results, feature importance, and model reload comparison.

4. Prototype interaction and implementation

The Streamlit app (app.py) accepts a CSV file with all 41 required feature columns. It validates column names and numeric values, loads the serialized pipeline, and displays predictions with probability scores. The user can download both the sample input and the prediction result CSV. The FastAPI service (api.py) provides /health, /schema and POST /predict; /docs exposes interactive API documentation.

- Prepare a trusted model artifact using: python train_model.py (NSL-KDD files must be in data/ or downloadable through KaggleHub).
- Install dependencies: pip install -r requirements.txt.
- Launch the interface: streamlit run app.py.
- Upload examples/sample_input.csv and select Predict intrusion class. The sample row is illustrative, not a live network event.
- For API use, start uvicorn api:app --reload and submit a JSON record containing all 41 feature fields to POST /predict.

Automated tests cover schema length and order, numeric conversion, optional metadata removal, invalid/missing fields, empty requests, prediction output formatting with a deterministic test double and API schema/validation behavior. They do not establish performance of the trained classifier. The local packaging run passed 10 tests. The real model artifact was not available in the supplied files; therefore end-to-end inference with the trained weights still requires a separate smoke test.

5. Deployment approach

GitHub is the public source-code and documentation home. It does not by itself execute predictions. The trained .joblib artifact must be created and published as a trusted, versioned repository asset or release asset, with its SHA-256 checksum. The Streamlit application can then be deployed from the public repository using Streamlit Community Cloud; alternatively the FastAPI endpoint can be hosted as a service. Docker is included as an optional reproducible packaging route.

Before release, confirm dependency compatibility between training and inference, ensure the model file is within repository hosting limits or use a release/artifact host, test in a signed-out browser, and record the actual public repository and app URLs. No URL is represented as live in this portfolio until verified.

6. Monitoring, support and maintenance

The monitoring plan is proposed, not active. A future controlled pilot should record request volume, service error rate, latency percentiles, CPU/memory, model version and predicted class distribution. Once analyst-verified labels exist, evaluate rolling per-class precision/recall/F1, confusion matrices and false-positive rates. Overall accuracy alone is insufficient for this imbalanced classification task.

Feature drift indicators such as PSI and KS tests can trigger investigation, but predicted labels are not ground truth. Retraining should follow verified data collection, label review, time/network-aware validation and an independent final evaluation. Track source code, dataset version, dependencies, random seed, hyperparameters, evaluation metrics, model checksum and approval. Test candidate models offline and in shadow mode before controlled rollout; maintain rollback capability.

Security practices include authorized and minimized telemetry, access controls, transport/storage encryption, retention policies, dependency review, request validation, rate limiting, sanitized logging, and loading only trusted

model artifacts. A checksum helps detect accidental corruption but is not a signature against a malicious publisher. High-impact alerts require analyst review. Snort/Suricata fallback, shadow deployment and automated retraining are future proposals, not current prototype features.

7. Dataset and synthetic-data statement

Training and evaluation use public NSL-KDD benchmark files, not a newly collected live operational traffic dataset. The sample CSV in the repository is an illustrative constructed input. SMOTENC generates synthetic minority-class examples within the training workflow only; validation and KDDTest+ are not oversampled. Raw benchmark files are not included in the repository. Dataset host terms and original attribution must be checked before redistributing benchmark data.

8. Limitations and future improvements

- The model generalizes substantially worse on the independent KDDTest+ set than on the KDDTrain+ validation split.
- R2L and U2R recall (0.06 and 0.10) is too low for dependable detection; many R2L events were classified as Normal.
- The NSL-KDD benchmark does not represent a current live enterprise network and does not prove detection of modern or zero-day attacks.
- The sample CSV is illustrative; live ingestion, packet capture, blocking, continuous monitoring, authentication, rate limiting, and operational fallback are not implemented.
- Future work should investigate distribution shift, rare-class error patterns, cost-sensitive learning, calibration, temporal/network-aware validation, newer datasets and controlled pilot data.

9. Group members and contribution record

Member	Individual work-package assignment
Adugna Necho Mulatu	To be confirmed by the group
Aminadab Tewhibo Mihret	To be confirmed by the group
Hussen Yasin Mohammed	To be confirmed by the group
Abdulfetah Hayerdin Mohammed	To be confirmed by the group
Nebiyu Mesfin Fikade	To be confirmed by the group

The supplied notebook provides the group roster but not individual role assignments. The members must replace the placeholders with accurate responsibilities and supporting evidence before final submission.

10. Submission package

The ZIP archive contains the source notebook, updated notebook export cell, application/API code, training script, sample CSV, tests, README, monitoring plan, evaluation/limitations note, synthetic-data statement, video script, team contribution sheet and one-page retrospective. The capstone video must be recorded by the group and kept within 10 minutes and 250 MB. Follow docs/submission_checklist.md before uploading to Canvas.

References

Tavallaee, M., Bagheri, E., Lu, W. and Ghorbani, A. A. (2009). “A Detailed Analysis of the KDD CUP 99 Data Set.” IEEE Symposium on Computational Intelligence for Security and Defense Applications.

Dataset source used by the notebook: KaggleHub dataset `hassan06/nslkdd`; a public NSL-KDD mirror is documented at https://github.com/jmnwong/NSL-KDD-Dataset. Check dataset redistribution terms before sharing raw files.